

Documentacion:

Objetivo

¡Hoy discutiremos un algoritmo de ordenamiento simple llamado **Bubble Sort** (Ordenamiento de Burbuja)! Consulta la pestaña de Tutoriales para acceder a los materiales de aprendizaje y al video instructivo.

Considera la siguiente versión de Bubble Sort:

C++

```
for (int i = 0; i < n; i++) {  
    // Rastrea el número de elementos intercambiados durante un solo recorrido del arreglo  
    int numberOfSwaps = 0;  
  
    for (int j = 0; j < n - 1; j++) {  
        // Intercambia elementos adyacentes si están en orden decreciente  
        if (a[j] > a[j + 1]) {  
            swap(a[j], a[j + 1]);  
            numberOfSwaps++;  
        }  
    }  
}  
  
// Si no se intercambiaron elementos durante un recorrido, el arreglo ya está ordenado  
if (numberOfSwaps == 0) {  
    break;  
}  
}
```

Tarea

Dado un arreglo `$a$` de tamaño `$n$` con elementos distintos, ordena el arreglo en orden ascendente utilizando el algoritmo de Bubble Sort mostrado arriba. Una vez ordenado, imprime las siguientes `$3$` líneas:

1. **Array is sorted in numSwaps swaps.**
(donde *numSwaps* es el número total de intercambios realizados)
2. **First Element: firstElement**
(donde *firstElement* es el primer elemento del arreglo ordenado)
3. **Last Element: lastElement**
(donde *lastElement* es el último elemento del arreglo ordenado)

Pista: Para completar este desafío, necesitarás agregar una variable que lleve el recuento total de todos los intercambios que ocurren durante la ejecución.

Ejemplo

Arreglo original `$a$`: `4 3 1 2`

- **Ronda 1** -> \$a\$: 3 1 2 4 | intercambios en esta ronda: \$3\$
- **Ronda 2** -> \$a\$: 1 2 3 4 | intercambios en esta ronda: \$2\$
- **Ronda 3** -> \$a\$: 1 2 3 4 | intercambios en esta ronda: \$0\$

En la primera ronda, el \$4\$ se intercambia en cada una de las \$3\$ comparaciones, terminando en la última posición. En la segunda ronda, el \$3\$ se intercambia en \$2\$ de las \$3\$ comparaciones. Finalmente, en la tercera ronda no se realizan intercambios, por lo que las iteraciones se detienen. La salida es la siguiente:

Plaintext

Array is sorted in 5 swaps.

First Element: 1

Last Element: 4

Formato de entrada

- La primera línea contiene un número entero, \$n\$, que indica el número de elementos en el arreglo \$a\$.
- La segunda línea contiene \$n\$ enteros separados por espacios que describen el arreglo \$a\$.

Restricciones

- \$2 \leq n \leq 600\$
- \$1 \leq a[i] \leq 2 \times 10^6\$, donde \$0 \leq i < n\$.

Formato de salida

Imprime las siguientes tres líneas de salida:

Plaintext

Array is sorted in numSwaps swaps.

First Element: firstElement

Last Element: lastElement

Ejemplo de Entrada 0

Plaintext

3

1 2 3

Ejemplo de Salida 0

Plaintext

Array is sorted in 0 swaps.

First Element: 1

Last Element: 3

Explicación 0

El arreglo ya está ordenado, por lo que se realizan \$0\$ intercambios e imprimimos las \$3\$ líneas de salida necesarias mostradas arriba.

Ejemplo de Entrada 1

Plaintext

3

3 2 1

Ejemplo de Salida 1

Plaintext

Array is sorted in 3 swaps.

First Element: 1

Last Element: 3

Explicación 1

El arreglo a no está ordenado, por lo que realizamos los siguientes 3 intercambios.

Tras 3 intercambios, el arreglo queda ordenado.